SOFTWARE ENGINEERING PROJECT PROPOSAL

E-Doc: Doctor Appointment System

A Web-Based Healthcare Appointment Management Platform

Project Name:	**E-Doc - Doctor Appointment System**
Course:	Software Engineering (SE)
Team Leader:	Syeda Kissa Zehra Naqvi
Team Member 1:	Afaq Ahmed Soomro
Team Member 2:	Amna Khan Awan
Teacher:	Engr. Abdul Rahman
Date:	February16, 2026

Table of Contents

1. **Overview**

2. **Project Detail**
   - **2.1** Problem or Challenge
   - **2.2** Project Vision
   - **2.3** Project Feasibility Study
     - **2.3.1** Technical Feasibility
     - **2.3.2** Operational Feasibility
     - **2.3.3** Economic Feasibility
   - **2.4** Project Goals
   - **2.5** Project Abstract
   - **2.6** Project Scope
   - **2.7** Project Team
   - **2.8** Existing & Related Work
   - **2.9** Proposed Solution & Architecture
   - **2.10** Use Case Model
   - **2.11** Feature List
   - **2.12** Software Development Methodology
   - **2.13** Proposed Infrastructure, Platform & Tools
   - **2.14** Project Timeline
   - **2.15** Mockups (Optional)
   - **2.16** References

3. **SE Project Improvements (Enhancements Over DB Project)**
   - **3.1** Software Architecture Improvements
   - **3.2** Security Enhancements
   - **3.3** Testing
   - **3.4** API & Modularity
   - **3.5** DevOps & CI/CD
   - **3.6** User Experience & Frontend
   - **3.7** Documentation & Code Quality

1. Overview

E-Doc is a comprehensive web-based Doctor Appointment System developed using PHP, HTML, CSS, and MySQL. Originally built as a Database (DB) course project, this proposal outlines the plan to extend and re-engineer it for the Software Engineering (SE) course, applying proper SE principles, design patterns, and development methodologies.

The system addresses the critical challenge of manual appointment booking in healthcare settings, allowing patients to register online, browse doctors by specialty, view schedules, and request appointments digitally. The system serves three key user roles: Admin, Doctor, and Patient.

For the SE course, the project will be enhanced with improved software architecture, security hardening, API-based communication, unit testing, CI/CD pipelines, and a modern front-end experience — transforming it from a functional DB prototype into a production-ready, maintainable software system.

The project is expected to be completed over one semester (~16 weeks) and is self-contained with no external project dependencies.

2. Project Detail

2.1 Problem or Challenge

In Pakistan and many developing regions, accessing healthcare begins with the frustrating challenge of booking a doctor appointment. Patients are often forced to call clinics during busy hours, stand in long queues, or rely on intermediaries, all of which introduce delays, miscommunication, and scheduling conflicts. This is especially problematic in semi-urban and rural areas where clinic resources are limited.

From the doctor's side, managing appointment slots manually across paper logs or basic spreadsheets is error-prone and inefficient. Double-bookings, missed sessions, and lack of patient history at the point of consultation are common pain points. There is a clear need for a centralized, easy-to-use digital solution that bridges this gap between patients and healthcare providers.

2.2 Project Vision

The vision of E-Doc is to democratize access to healthcare appointments by providing an intuitive, reliable, and secure web platform that seamlessly connects patients with doctors. The system aims to eliminate paper-based scheduling, reduce wait times, and empower both patients and healthcare providers with a transparent, real-time appointment management experience.

Upon completion, E-Doc will serve as a scalable foundation for digital healthcare management — one that can be adopted by local clinics, hospitals, and independent practitioners. The SE-enhanced version will demonstrate professional software engineering practices including layered architecture, test coverage, and clean code, making it suitable for real-world deployment and future expansion.

2.3 Project Feasibility Study

A feasibility study was conducted to evaluate whether the E-Doc project can realistically be developed and deployed given the team's resources and timeline.

2.3.1 Technical Feasibility

The existing DB project already demonstrates that the core technical stack (PHP, MySQL, HTML/CSS) is well within the team's capabilities. The SE enhancements — including MVC architecture, REST APIs, PHPUnit testing, and Docker containerization — are achievable using widely documented, open-source tools. The team has access to GitHub for version control, and the project already includes a Dockerfile, demonstrating early readiness for containerized deployment.

2.3.2 Operational Feasibility

The system is designed to be user-friendly for all three roles (Admin, Doctor, Patient) and requires only a web browser for access no special hardware or software installation is needed for end users. XAMPP or Docker can be used for local development and testing. The team can realistically operate and test the system in a local development environment before staging deployment.

2.3.3 Economic Feasibility

The project uses entirely free and open-source technologies: PHP, MySQL, Apache (via XAMPP), Bootstrap, and PHPMailer. Hosting can be done for free on platforms like InfinityFree or via Docker on a

cloud free tier. There are no licensing costs involved, making this a zero-budget project with high economic viability.

2.4 Project Goals

The project goals are categorized as follows:

Functional Goals:

- Enable patients to register, browse doctors by specialty, and book appointments online
- Allow doctors to view, manage, and confirm appointment requests
- Provide an admin panel to manage doctors, sessions, and patient records
- Implement email notifications via PHPMailer for appointment confirmations

Technological Goals:

- Refactor codebase to follow MVC (Model-View-Controller) architectural pattern
- Introduce RESTful API layer (existing /api folder) with proper routing and validation
- Containerize application using Docker for consistent deployment
- Implement PHPUnit for automated unit and integration testing

Quality Goals:

- Achieve at least 70% code coverage with automated tests
- Enforce input validation and prepared statements to prevent SQL injection
- Apply OWASP security best practices for authentication and session management

Organizational Goals:

- Apply Agile/Scrum methodology with 2-week sprints
- Use GitHub Projects for issue tracking and pull request-based code reviews
- Demonstrate proper use of SE principles learned in the SE course

2.5 Project Abstract

E-Doc is a web-based Doctor Appointment Booking System designed to streamline the process of scheduling medical consultations. Built using PHP and MySQL, the system supports three distinct user roles: Administrator, Doctor, and Patient. Admins can manage doctor profiles, set available sessions, and monitor bookings. Doctors can view their scheduled appointments and patient details. Patients can register, search for doctors by specialty, review weekly availability, and submit appointment requests.

Originally developed as a Database course project, E-Doc is being re-engineered for the Software Engineering course to apply SE best practices. The improvements include adopting the MVC design pattern, implementing a RESTful API, adding automated testing with PHPUnit, enforcing security standards, and establishing a CI/CD-ready development workflow using Docker and GitHub Actions. The final deliverable will be a maintainable, well-documented, and testable web application that demonstrates practical application of software engineering principles in a real-world healthcare domain.

2.6 Project Scope

In Scope:

- Patient registration, login, and profile management

- Doctor browsing and filtering by specialty
- Appointment request, confirmation, and history viewing
- Admin management of doctors, sessions, and bookings
- Doctor schedule management and patient appointment viewing
- Email notifications via PHPMailer
- MVC refactoring and REST API development
- Automated unit testing and Docker containerization

Out of Scope:

- Mobile native app (iOS/Android)
- Online payment or insurance processing
- Telemedicine / video consultation features
- Integration with external hospital management systems
- Multi-language support

2.7 Project Team

Name	Role	Availability	Responsibility
<Team Leader Name>	Project Lead / Backend Dev	Full-time (SE Course)	Architecture, API, Sprint Planning
<Member 2>	Frontend Developer	Full-time (SE Course)	UI/UX, HTML/CSS, Mockups
<Member 3>	Database / Testing	Full-time (SE Course)	DB optimization, PHPUnit Tests
<Member 4>	DevOps / Documentation	Full-time (SE Course)	Docker, GitHub Actions, Docs

2.8 Existing & Related Work

Below is a comparison of E-Doc with similar existing systems:

System	Technology	Role-based Access	Open Source	Unique Feature
Practo	Web/Mobile (React, Node.js)	Yes	No	Doctor reviews & ratings
Zocdoc	Web/Mobile (Ruby, AWS)	Yes	No	Real-time slot availability
OpenEMR	PHP, MySQL	Yes	Yes	Full EMR with billing
HMS (Generic)	PHP, MySQL	Partial	Yes	In-patient management
E-Doc (Ours)	PHP, MySQL, Docker	Yes (3 roles)	Yes	Lightweight, containerized, SE-focused

E-Doc differentiates itself through its lightweight architecture, full open-source availability, Docker support, and its purpose as a learning-oriented SE project that can be adopted by small clinics with minimal infrastructure.

2.9 Proposed Solution & Architecture

The system will follow a 3-tier MVC architecture:

Layer	Component	Technology
Presentation (View)	Patient Portal, Doctor Dashboard, Admin Panel	HTML5, CSS3, Bootstrap 5, JavaScript
Business Logic (Controller)	Appointment Controller, Auth Controller, Admin Controller	PHP 8 (MVC Controllers)
Data Access (Model)	User Model, Doctor Model, Appointment Model	PHP PDO + MySQL
API Layer	REST API endpoints (/api)	PHP JSON API
Infrastructure	Web Server, Database, Container	Apache 2.4, MySQL 5.7, Docker
Communication	Email Notifications	PHPMailer (SMTP)

Architecture flow: Patient/Doctor/Admin → Browser → Apache Web Server → PHP MVC Controllers → Models (PDO) → MySQL Database. The /api directory exposes JSON endpoints for potential future mobile/SPA clients.

2.10 Use Case Model

The system involves three primary actors with the following key use cases:

Actor: Patient

- Register / Login / Logout
- Search Doctors by Specialty
- View Doctor Weekly Schedule
- Book Appointment
- View Appointment History
- Edit / Delete Account

Actor: Doctor

- Login / Logout
- View Scheduled Sessions
- View All Appointments
- View Patient Details
- Edit / Delete Account

Actor: Admin

- Add / Edit / Delete Doctors
- Create / Remove Doctor Sessions
- View All Patient Records
- View All Bookings
- Manage System Users

2.11 Feature List

The following 20 core domain features are planned for the SE version of E-Doc:

#	Feature	Actor
1	Patient Online Registration	Patient
2	Doctor Search by Specialty	Patient
3	View Doctor Weekly Availability	Patient
4	Submit Appointment Request	Patient
5	View Personal Appointment History	Patient
6	Appointment Confirmation Email Notification	Patient/System
7	Doctor Session Scheduling by Admin	Admin
8	Add / Edit / Delete Doctor Profiles	Admin
9	View All Patient Records	Admin
10	View and Monitor All Bookings	Admin
11	Doctor Appointment Viewing	Doctor
12	Patient Detail Access by Doctor	Doctor
13	Doctor Schedule Management	Doctor
14	Appointment Request Approval/Rejection	Doctor
15	Patient Account Self-Management	Patient
16	Doctor Account Self-Management	Doctor
17	Specialty-Based Doctor Filtering	Patient/Admin
18	Admin Session Removal	Admin
19	Booking Cancellation by Patient	Patient
20	Audit Log for Admin Actions	Admin/System

2.12 Software Development Methodology

The team will use Agile Scrum methodology with 2-week sprints over the course semester. This choice is justified because:

- The project involves iterative enhancement of an existing codebase, which suits Agile's incremental delivery model

- Requirements may evolve as the SE course progresses and instructor feedback is received
- Short sprints allow the team to demo working features regularly

Sprint Structure:

Sprint	Duration	Key Deliverables
Sprint 0 (Planning)	Week 1-2	Requirement analysis, environment setup, DB schema review, GitHub setup
Sprint 1 (Architecture)	Week 3-4	MVC refactoring, folder restructuring, routing implementation
Sprint 2 (Core Features)	Week 5-6	Patient registration, doctor search, appointment booking
Sprint 3 (Admin & Doctor)	Week 7-8	Admin panel, doctor dashboard, session management
Sprint 4 (API & Security)	Week 9-10	REST API endpoints, input validation, SQL injection prevention
Sprint 5 (Testing)	Week 11-12	PHPUnit tests, integration tests, bug fixing
Sprint 6 (DevOps & Polish)	Week 13-14	Docker CI/CD, code review, documentation
Sprint 7 (Final)	Week 15-16	Final testing, deployment, presentation prep

2.13 Proposed Infrastructure, Platform & Tools

Category	Tools / Technologies
Backend Language	PHP 8.x
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript
Database	MySQL 5.7 / MariaDB
Web Server	Apache 2.4 (XAMPP)
Email Library	PHPMailer 6.x
Containerization	Docker, Docker Compose
Version Control	Git, GitHub
Testing Framework	PHPUnit 10.x
Project Management	GitHub Projects / Issues
IDE	VS Code / PhpStorm
API Testing	Postman
Mockup / Design	Balsamiq / Figma
CI/CD	GitHub Actions

Category	Tools / Technologies
Documentation	Markdown, Word

2.14 Project Timeline

The project follows a 16-week Agile Scrum timeline aligned with the SE course semester:

Week	Milestone / Activity	Deliverable
1-2	Project setup, requirement gathering, GitHub repo init	Project Proposal Document
3-4	MVC architecture refactoring, routing setup	Refactored codebase + PR review
5-6	Patient module: registration, doctor search, booking	Working patient portal
7-8	Doctor & Admin modules, session management	Functional admin + doctor dashboards
9-10	REST API development, security hardening	API endpoints + security report
11-12	Unit & integration testing with PHPUnit	Test suite with 70%+ coverage
13-14	Docker setup, GitHub Actions CI/CD pipeline	Dockerized app + CI pipeline
15-16	Final QA, documentation, deployment, presentation	Final release + SRS Document

2.15 Mockups (Optional)

Mockups will be designed using Balsamiq and will cover the following key screens:

- Patient Registration & Login Page
- Doctor Search & Specialty Filter Page
- Doctor Profile & Weekly Schedule View
- Appointment Booking Confirmation Screen
- Admin Dashboard with Doctor & Booking Management

Mockup tool: Balsamiq (https://balsamiq.com/products/mockups/). Full mockup screens will be included in the SRS document.

2.16 References

[1] R. Pressman and B. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill, 2019.

[2] IEEE, "IEEE Reference Guide," IEEE Author Center, 2022. [Online]. Available: https://ieeeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf

[3] PHPMailer, "PHPMailer Documentation," GitHub, 2023. [Online]. Available: https://github.com/PHPMailer/PHPMailer

[4] OWASP Foundation, "OWASP Top Ten," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[5] K. Beck et al., "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org/

[6] Docker Inc., "Docker Documentation," 2024. [Online]. Available: https://docs.docker.com/

3. SE Project Improvements (Enhancements Over DB Project)

The following section outlines the specific improvements planned to transform the E-Doc DB course project into a mature SE project:

3.1 Software Architecture Improvements

- Refactor from procedural PHP to MVC (Model-View-Controller) pattern, separating concerns properly
- Introduce a proper routing mechanism (e.g., .htaccess + front controller) instead of direct file access
- Create a dedicated /models, /controllers, /views folder structure
- Introduce a service layer for business logic (e.g., AppointmentService, NotificationService)

3.2 Security Enhancements

- Replace raw MySQL queries with PDO Prepared Statements to prevent SQL injection
- Implement password hashing using PHP password_hash() (bcrypt: removes plain-text or MD5 passwords)
- Add CSRF token protection on all forms
- Implement session security: regenerate session IDs on login, set HTTPOnly and Secure cookie flags
- Add input sanitization and server-side validation on all user inputs
- Upgrade to PHP 8.x for improved type safety and security patches

3.3 Testing

- Introduce PHPUnit for unit and integration testing of all models and controllers
- Write test cases for appointment booking logic, user authentication, and admin operations
- Target minimum 70% code coverage
- Add end-to-end test scenarios for the three user roles

3.4 API & Modularity

- Expand the existing /api directory into a proper RESTful API (JSON responses, HTTP status codes)
- Document API endpoints using OpenAPI/Swagger specification
- Decouple frontend views from backend logic to prepare for future SPA or mobile clients

3.5 DevOps & CI/CD

- Enhance the existing Dockerfile and docker-compose.yml for multi-environment support (dev/prod)
- Set up GitHub Actions CI/CD pipeline: run tests on every push, lint PHP code, auto-deploy on merge to main

- Use environment variables (.env) for database credentials and sensitive config (remove hardcoded values)

3.6 User Experience & Frontend

- Upgrade UI to Bootstrap 5 with a responsive, mobile-friendly design
- Implement AJAX-based appointment booking for a smoother user experience (no full page reload)
- Add a real-time doctor availability calendar view
- Improve form validation with client-side feedback using JavaScript

3.7 Documentation & Code Quality

- Add PHPDoc comments to all classes and methods
- Create a comprehensive README.md with setup instructions, architecture overview, and API docs
- Follow PSR-12 PHP coding standards
- Generate a full SRS (Software Requirements Specification) document
- Maintain a changelog and versioned releases on GitHub